SQL 3 (c.d.)

DELETE + join

```
DELETE FROM films USING producers
  WHERE producer_id = producers.id AND producers.name = 'foo';
```

```
DELETE FROM films
  WHERE producer_id IN (SELECT id FROM producers WHERE
name = 'foo');
```

DELETE + join

```
DELETE FROM films USING producers
  WHERE producer_id = producers.id AND producers.name = 'foo';
```

```
DELETE FROM films
  WHERE producer_id IN (SELECT id FROM producers WHERE
name = 'foo');
```

TRANSACTIONS

```
BEGIN;
UPDATE accounts
SET balance = balance - 100.00
    WHERE name = 'Alice';
-- etc etc
COMMIT;
```

```
BEGIN;
UPDATE accounts
SET balance = balance - 100.00
    WHERE name = 'Alice';
-- etc etc
ROLLBACK;
```

TRANSACTIONS

```
BEGIN;
UPDATE accounts
SET balance =
balance - 100.00
    WHERE name = 'Alice';
-- etc etc
COMMIT;
```

```
BEGIN;
UPDATE accounts
SET balance = balance - 100.00
    WHERE name = 'Alice';
-- etc etc
ROLLBACK;
```

without BEGIN -> AUTOCOMMIT
(i.e., implicit COMMIT if success, ROLLBACK otherwise)

Window Functions

```
SELECT depname, empno, salary,
    avg(salary) OVER (PARTITION BY depname) FROM empsalary;
```

Window Functions

A window function performs a calculation across a set of table rows that are somehow related to the current row.

Window Functions

A window function performs a calculation across a set of table rows that are somehow related to the current row.

Behind the scenes, the window function is able to access more than just the current row of the query result.

Window Functions

```
SELECT depname, empno, salary,
    avg(salary) OVER (PARTITION BY depname) FROM empsalary;
```

- - average taken across all the table rows that have the same depname value as the current row

Window Functions

```
SELECT depname, empno, salary,
    avg(salary) OVER (PARTITION BY depname) FROM empsalary;

 depname   | empno | salary |          avg
-----------+-------+--------+----------------------
 develop   |    11 |   5200 | 5020.0000000000000000
 develop   |     7 |   4200 | 5020.0000000000000000
 develop   |     9 |   4500 | 5020.0000000000000000
 develop   |     8 |   6000 | 5020.0000000000000000
 develop   |    10 |   5200 | 5020.0000000000000000
 personnel |     5 |   3500 | 3700.0000000000000000
 personnel |     2 |   3900 | 3700.0000000000000000
 (...)
```

Window Functions

```
SELECT depname, empno, salary,
       rank() OVER (PARTITION BY depname ORDER BY salary DESC)
FROM empsalary;
```

depname	empno	salary	rank
develop	8	6000	1
develop	10	5200	2
develop	11	5200	2
develop	9	4500	4
develop	7	4200	5
personnel	2	3900	1
personnel	5	3500	2

Window Functions

```
SELECT salary, sum(salary) OVER () FROM empsalary;
```

a single partition

Window Functions

```
SELECT salary, sum(salary) OVER () FROM empsalary;

salary  |  sum
--------+-------
   5200 | 47100
   5000 | 47100
   3500 | 47100
   4800 | 47100
   3900 | 47100
   4200 | 47100
   4500 | 47100
   4800 | 47100
   6000 | 47100
   5200 | 47100
```

Window Functions

```
SELECT salary, sum(salary) OVER (ORDER BY salary) FROM empsalary;
```

If ORDER BY is supplied then the frame consists of all rows from the start of the partition up through the current row, plus any following rows that are equal to the current row according to the ORDER BY clause.

Window Functions

```
SELECT salary, sum(salary) OVER (ORDER BY salary) FROM empsalary;

salary |  sum
--------+-------
   3500 |  3500
   3900 |  7400
   4200 | 11600
   4500 | 16100
   4800 | 25700
   4800 | 25700
   5000 | 30700
   5200 | 41100
   5200 | 41100
   6000 | 47100
```

Window Functions

```
SELECT sum(salary) OVER w, avg(salary) OVER w
  FROM empsalary
  WINDOW w AS (PARTITION BY depname ORDER BY salary DESC);
```

Window Functions

- permitted only in SELECT list / ORDER BY clause
- forbidden in GROUP BY, HAVING and WHERE clauses
  - logically executed after them
- executed after non-window aggregate functions
  - it is valid to include an aggregate function call in the arguments of a window function (but not vice versa).

Docs:

Intro to window functions
Window function list

Nowa składnia (pipelining)

SQL Has Problems. We Can Fix Them: Pipe Syntax In SQL | Proceedings of the VLDB Endowment

Nowa składnia (pipelining)

- SQL’s foundational semantics, from relational algebra, are excellent.
- SQL’s conceptual data model and top-level syntax, with statements representing queries, DDL, DML, etc, and composability via subqueries, works well.
- SQL’s basic operations within a query (clauses like JOIN, ORDER BY, etc), all work reasonably well.
- SQL’s syntactic structure for composing queries using those operations is terrible.
- SQL’s localized syntax with English-like keyword phrases is an anachronism, but we can live with it.
- SQL’s expression language is fine, and implementations typically include a good library of existing functions.

Nowa składnia (pipelining)

- SQL's foundational semantics, from relational algebra, are excellent.
- SQL's conceptual data model and top-level syntax, with statements representing queries, DDL, DML, etc, and composability via subqueries, works well.
- SQL's basic operations within a query (clauses like JOIN, ORDER BY, etc), all work reasonably well.
- **SQL's syntactic structure for composing queries using those operations is terrible.**
- SQL's localized syntax with English-like keyword phrases is an anachronism, but we can live with it.
- SQL's expression language is fine, and implementations typically include a good library of existing functions.

Nowa składnia (pipelining)

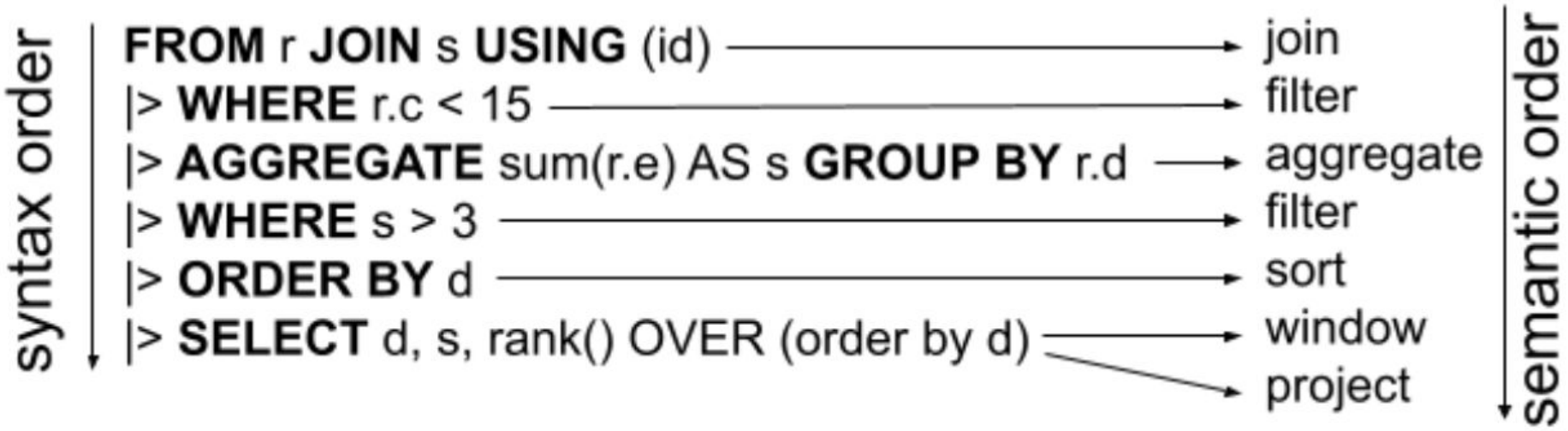
syntax order

```
SELECT r.d,
       sum(r.e) as s,
       rank() over (order by r.d)
FROM r, s
WHERE r.a = s.b
  AND r.c < 15
GROUP BY r.d
HAVING sum(r.e) > 3
ORDER by r.d
```

join
filter
aggregation
filter
window
sort
project

semantical order

Nowa składnia (pipelining)



Nowa składnia (pipelining)

```
<query> :=
   {all existing query syntaxes}
   | "FROM" <from_body>                -- New: FROM as a query
   | <query> "|>" <pipe_operator>      -- New: pipe suffixes

<pipe_operator> :=
   "WHERE" <expression>
   | "ORDER" "BY" <order_by_body>
   | "JOIN" <table_expression> [["AS"] alias]
   ...
```

Nowa składnia (pipelining)

```
FROM orders
|> SET o_orderstatus = LOWER(o_orderstatus)
|> EXTEND ROUND(o_totalprice) AS dollars
|> DROP o_comment
|> AS result
|> SELECT o_custkey, result.dollars
```

Operator	Output (modified rows and columns)
`SELECT <expr> [[AS] alias], ...`	Same rows, with exactly these selected columns. `DISTINCT`, `*` and other common `SELECT` modifiers are also allowed here.
`EXTEND <expr> [[AS] alias], ...`	Same rows, with added columns.
`SET <column> = <expression>, ...`	Same rows, with updated values for modified columns.
`DROP <column>, ...`	Same rows, minus dropped columns.
`AS <alias>`	Same rows and columns, but with a new table alias available. Old table aliases are removed.
`WHERE <condition>`	Subset of input rows passing condition.
`LIMIT <n> [OFFSET <m>]`	Subset of input rows subject to counts, respecting order if input table was ordered.
`AGGREGATE <agg_expr> [[AS] alias], ...`	Full-table aggregation, with one row, with a column for each aggregate expression.
`AGGREGATE [<agg_expr> [[AS] alias], ...]` `GROUP BY <grouping_expr> [AS alias], ...`	Aggregation with grouping, with one row per group. The column list has the grouping columns and then the aggregate columns. Aliases can be assigned directly on grouping expressions.
`[LEFT|...] JOIN {table/subquery/...}` `[ON <condition> | USING(col, ...)]`	Join results, with a filtered cross-product of the pipe input table and the table expression following `JOIN`.
`ORDER BY <expr> [ASC|DESC], ...`	Same rows but sorted.
`{UNION|INTERSECT|EXCEPT} {ALL|DISTINCT}` `(<query>), (<query>), ...`	Set operation output, including combined rows from the input table plus *one or more* tables passed as arguments.
`CALL tvf(args, ...)`	Table-valued function output rows and columns. The pipe input table is passed to the TVF as its first table-typed argument.
`TABLESAMPLE <method> (<size> {ROWS|PERCENT})`	Subset of rows produced by the chosen sampling algorithm.
`PIVOT (agg_expr FOR col IN (value1, ...))`	New table with rows pivoted to become columns.
`UNPIVOT (value_col FOR key_col IN (col1, ...))`	New table with columns pivoted to become rows.

Nowa składnia (pipelining)

```
FROM orders
|> AGGREGATE SUM(o_totalprice) AS price, COUNT(*) AS cnt
   GROUP BY EXTRACT(year FROM o_orderdate) AS year
```

Nowa składnia (pipelining)

```sql
SELECT c_count, COUNT(*) AS custdist
FROM
  ( SELECT c_custkey, COUNT(o_orderkey) c_count
    FROM customer
    LEFT OUTER JOIN orders ON c_custkey = o_custkey
         AND o_comment NOT LIKE '%unusual%packages%'
    GROUP BY c_custkey
  ) AS c_orders
GROUP BY c_count
ORDER BY custdist DESC, c_count DESC;
```

Nowa składnia (pipelining)

```
FROM customer
|> LEFT OUTER JOIN orders ON c_custkey = o_custkey
      AND o_comment NOT LIKE '%unusual%packages%'
|> AGGREGATE COUNT(o_orderkey) c_count
   GROUP BY c_custkey
|> AGGREGATE COUNT(*) AS custdist
   GROUP BY c_count
|> ORDER BY custdist DESC, c_count DESC;
```

Nowa składnia (pipelining)

- Against SQL
- A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language
- SQL Has Problems. We Can Fix Them: Pipe Syntax In SQL | Proceedings of the VLDB Endowment
- Pipe query syntax | BigQuery | Google Cloud
- SQL Pipeline Syntax | Databricks Documentation

GROUPING SETS

```
=> SELECT * FROM items_sold;
 brand | size | sales
-------+------+-------
 Foo   | L    |  10
 Foo   | M    |  20
 Bar   | M    |  15
 Bar   | L    |  5
(4 rows)
```

GROUPING SETS

```
SELECT * FROM items_sold;
```

```
 brand | size | sales
-------+------+-------
 Foo   | L    |  10
 Foo   | M    |  20
 Bar   | M    |  15
 Bar   | L    |  5
(4 rows)
```

```
SELECT brand, size, sum(sales)
 FROM items_sold GROUP BY
   GROUPING SETS ((brand), (size),());
```

```
 brand | size | sum
-------+------+-----
  Foo  |      |  30
  Bar  |      |  20
       | L    |  15
       | M    |  35
       |      |  50
(5 rows)
```

GROUPING SETS

```
ROLLUP ( e1, e2, e3, ... )

is equivalent to

GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)
```

GROUPING SETS

```
ROLLUP ( e1, e2, e3, ... )

is equivalent to

GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)
```

- analysis over hierarchical data; e.g., total salary by department, division, and company-wide total

GROUPING SETS

```
=> SELECT * FROM items_sold;

 make  | model | sales
-------+-------+-------
 Foo   | GT    |  10
 Foo   | Tour  |  20
 Bar   | City  |  15
 Bar   | Sport |  5
(4 rows)
```

GROUPING SETS

```
CUBE ( a, b, c )


is equivalent to

GROUPING SETS (
    ( a, b, c ),
    ( a, b    ),
    ( a,    c ),
    ( a       ),
    (    b, c ),
    (    b    ),
    (       c ),
    (         )
)
```

Dziękuję